

1. Introduction

The Java Quiz Application is a console-based program developed using Java. It allows users to answer multiple-choice questions, receive instant feedback, and view their final score. The project demonstrates basic object-oriented programming concepts through a multi-class structure.

2. Problem Statement

Manual quizzes require effort to check answers and calculate scores. This project provides an automated quiz system that displays questions, accepts user answers, validates input, checks correctness, and calculates the final result.

3. Functional Requirements

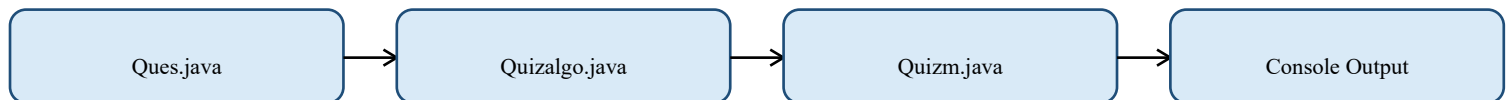
- Display a welcome message.
- Display multiple-choice questions with four options.
- Accept answers from the user.
- Validate entered options.
- Check answers and display feedback.
- Calculate and display score and percentage.

4. Non-Functional Requirements

- Simple and user-friendly console interface.
- Fast response after each answer.
- Reliable input handling.
- Easy to maintain and extend.
- Requires minimal system resources.
- Compatible with JDK 8 or higher.

5. System Architecture

The application follows a simple layered structure. The question class stores question data, the quiz logic class supports processing, and the main quiz class controls the user interaction and result display. [High-Level Architecture](#)

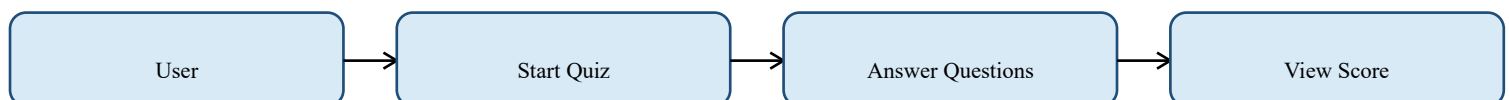


6. Design Diagrams

6.1 Use Case Diagram

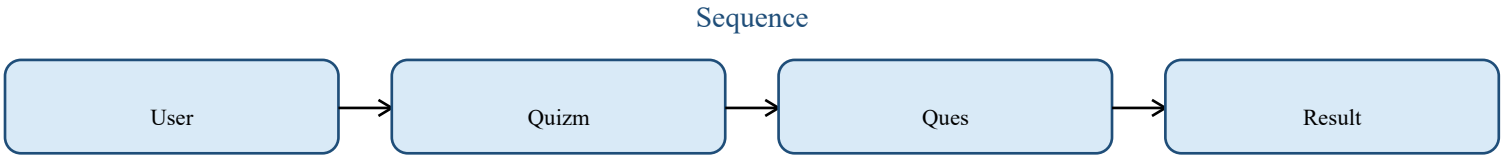
The user interacts with the quiz application by starting the quiz, answering questions, and viewing the final score.

Use Case Flow



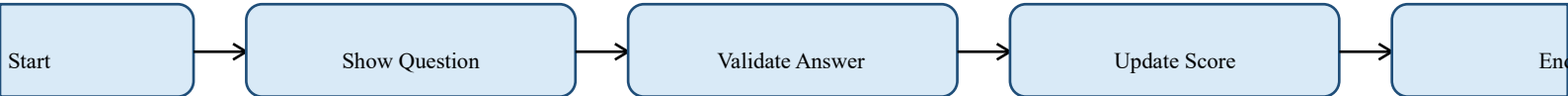
6.2 Sequence Diagram

The following sequence shows the basic interaction between the user and the application.



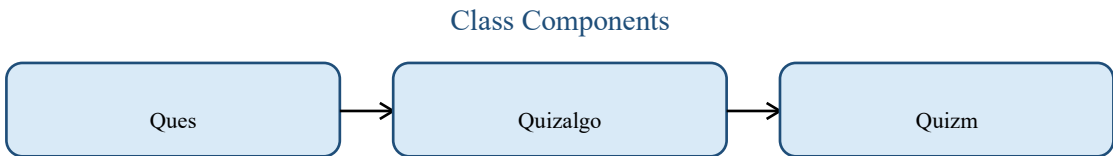
6.3 Class / Component Diagram

Ques: Stores question text, options, and correct answer.



Quizalgo: Handles quiz-related logic.

Quizm: Controls quiz flow, input, feedback, and score calculation.



7. Design Decisions & Rationale

- A multi-class structure was selected to separate responsibilities.
- Encapsulation is used to keep question data organized.
- Console input was selected because it is simple and suitable for a beginner Java project.
- Questions are stored in the program for easy testing and modification.
- Input validation improves reliability and user experience.

8. Implementation Details

Ques.java: Represents an individual question, stores options and the correct answer, displays the question, and checks the user's response.

Quizalgo.java: Contains quiz-related processing and logic.

Quizm.java: Initializes questions, displays the welcome message, runs the quiz loop, converts A/B/C/D into indexes, and displays the final score.

Score Formula: $\text{Percentage} = (\text{Correct Answers} / \text{Total Questions}) \times 100$.

9. Screenshots / Results

```
=====
Welcome to the Quiz!
=====

1. What is the capital of India?
A. New Delhi
B. Mumbai
C. Kolkata
D. Chennai
Enter your answer (A/B/C/D): a
? Correct answer!

2. What is 2 + 8?
A. 10
B. 12
C. 14
D. 16
Enter your answer (A/B/C/D): a
? Correct answer!

3. What is the largest planet in our solar system?
A. Earth
B. Jupiter
C. Saturn
D. Mars
Enter your answer (A/B/C/D): a
? Incorrect answer! The correct answer is: B. Jupiter

4. Earth is the third planet from the Sun. True or False?
A. True
B. False
Enter your answer (A/B/C/D): v
? Invalid input! Please enter A, B, C, or D.
4. Earth is the third planet from the Sun. True or False?
A. True
B. False
Enter your answer (A/B/C/D): d
? Incorrect answer! The correct answer is: A. True

=====
Quiz Complete!
=====
Your score: 2 out of 4
Percentage: 50.00%
=====

PS D:\STUDY\java-n\quizzz>
```

10. Testing Approach

Test Case	Input	Expected Result
Correct answer	A/B/C/D	Correct feedback and score update
Incorrect answer	Wrong option	Incorrect feedback
Invalid input	E or number	Ask user to enter a valid option
Complete quiz	All questions answered	Display final score and percentage

11. Challenges Faced

- Organizing the application into separate Java classes.
- Handling invalid user input.
- Converting character answers into numeric indexes.
- Maintaining correct score calculation.
- Designing clear console output.

12. Learnings & Key Takeaways

- Improved understanding of Java syntax and program structure.
- Learned practical use of classes and objects.
- Understood encapsulation and methods.
- Practiced loops, conditions, arrays, and input handling.
- Learned the basic stages of software development and testing.

13. Future Enhancements

- Add a graphical user interface using Swing or JavaFX.
- Add more questions and quiz categories.
- Randomize questions and options.
- Add timers and difficulty levels.
- Store scores using a database or file.
- Add user login and a leaderboard.

14. References

1. Java Documentation: <https://docs.oracle.com/en/java/>
2. Java Platform, Standard Edition Documentation.
3. Object-Oriented Programming class notes.
4. Project README and source code.